EXPANDING MOSTEK'S F8 EXTERNAL INTERRUPT CAPABILITIES

Expanding Mostek's F8 External Interrupt Capabilities

by Jim Vittera

INTRODUCTION

One of the considerations involved in the design of any microprocessor based system is how to structure the interface between the peripherals (inputs or devices being controlled) and the CPU. The data line interface is usually dictated by the peripheral itself (e.g., a paper tape reader is eight bits of parallel data, a teletype is two lines of serial data, and a switch or front panel lamp usually only requires one line of data). The control lines of these peripherals however, can be handled in one of two basic ways by the system designer. The first method of handling these control lines, which is probably the most common, is to have the CPU periodically scan the control lines (connected to a I/O Port) to see if they require service. This is done by a small program which inputs the control lines through an I/O Port into the accumulator. They are then tested to determine if a line is active and the program flow diverted to service the active control line. The second method is to allow these control lines to interrupt the processor and divert program flow to service that peripheral. Servicing of these control inputs in a F8 based system is the topic of this application note with particular emphasis placed on implementing interrupt driven systems.

SCANNED VS INTERRUPT DRIVEN SYSTEMS

The basic difference between scanned and interrupt driven systems is that in a scanned system the peripherals are checked periodically to see if they need service. This periodic interval can be determined by the count down of a hardware timer (a software timer could be used, but the CPU would be tied up implementing a ripple counter—not a very effective use of the microprocessor). This technique is good for peripherals which can wait for service by the CPU (the maximum time would be the time between counter outputs), and good examples are any peripheral activated or observed by a human. For example a keyboard/display might be scanned at 1 ms intervals, as determined by the timer, which would be slow by microprocessor standards but exceedingly fast by human standards (after pressing a key or throwing a switch an extra 1 ms delay in service would not be noticeable).

FLOWCHART FOR SCANNING N CONTROL LINES

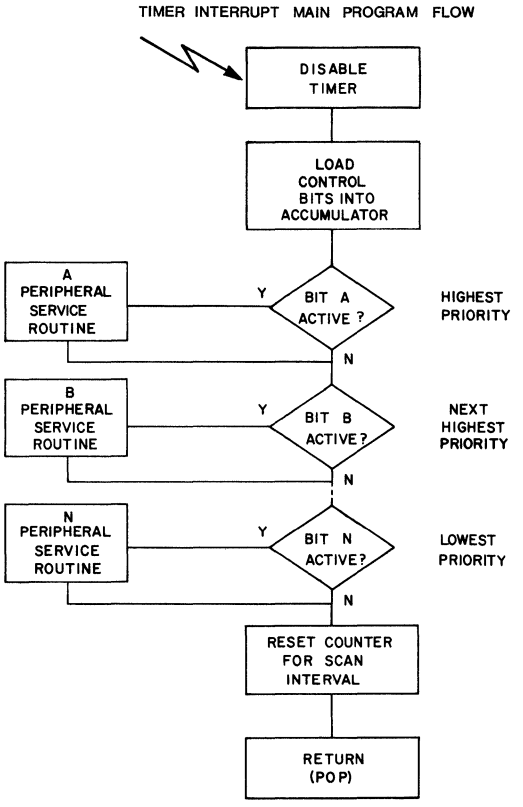


Figure 1

On the other hand many microprocessors are involved in the control of fast peripherals (Floppy Disk) or real time systems where quick response by the processor is required. In these situations, interrupt driven systems are mandatory, because the processor can be diverted from its present task to service the interrupting device in the order of tens of microseconds. Scanned systems are usually perferred by the system designer because they usually require less hardware, especially when implemented in a F8 System with its hardware timers. Figure 1 is a flow chart of a scanned system where the interval between scans is determined by the value preset into the timer. Note that priority is established by the order in which the control bits are tested and can be changed entirely by software.

HARDWARE VECTORED INTERRUPTS

The interrupt technique used by F8 Family devices capable of interrupting the CPU (PSU, PIO, or SMI) is to have the interrupting device provide to the CPU a Interrupt Vector unique to that interrupt. The CPU then loads this vector directly into the program

counter (saving the previous program counter in P) directing the CPU to the service routine for this interrupt. This technique provides a fast response to the interrupt because no time is consumed in polling to locate the interrupting device. In addition to providing automatic vectoring of the interrupts, the F8 devices provide automatic prioritizing of the interrupts. Priority is determined by the placement of the interrupting device in a daisy chain structure — a location closer to the CPU means higher priority — as shown in Figure 3. $\overline{ICB}$ is an output from the F8 CPU to indicate if interrupts have been enabled by the use of an EI instruction in the program being executed. $\overline{ICB}$ goes low when interrupts have been enabled, thereby enabling the daisy chain of interrupting devices. One or more of the three $\overline{EXT\ INT}$ inputs shown goes low signaling a request(s) for service by one or more of the peripherals. The device or devices that have $\overline{EXT\ INT}$ low now pull their $\overline{INT\ REQ}$ line low (assuming interrupts are not disabled at the local level) signaling the processor to begin an interrupt service sequence. The status of the $\overline{INT\ REQ}$ line is tested by the CPU at the end of every instruction which is not privileged. Privileged instructions cannot be interrupted so the CPU waits until the end of the next instruction (which is not privileged) to test the $\overline{INT\ REQ}$ line. When the CPU finds the $\overline{INT\ REQ}$ line low it begins the interrupt sequence by saving the Program Counter in P and using the ROM Control Lines to command the interrupting peripheral to transfer its vector address to the Program Counter. The 3851 is the highest priority device in Figure 3 and if its $\overline{EXT\ INT}$ line is low it sets its $\overline{PRI\ OUT}$ signal high thereby disabling all lower priority devices and outputs its vector address on the Data Bus. Should the PSU not be the interrupting device, it leaves its $\overline{PRI\ OUT}$ signal low passing the request to the second device in the chain (the PIO in this case). If the PIO is interrupting, it raises its $\overline{PRI\ OUT}$ line to a logic one and outputs its vector address. $\overline{PRI\ OUT}$ going high prevents all devices of lower priority from outputing their vector address even though they may be trying to interrupt. Twenty two cycles of the Φ clock are required to complete this interrupt vector fetch sequence. The next event that occurs is an instruction fetch from the location specified from the vector address. The SMI doesn't have a $\overline{PRI\ OUT}$ signal therefore it must be the lowest priority device in the system. The time required to get to an interrupt service routine can be calculated as shown in Figure 2 (at a 2 MHz Φ rate).

The time from an interrupt striking to the start of execution of its service routine is highly dependent on the instruction being executed at the time of the interrupt. The maximum number was based on a long privileged instruction such as PI followed by a long non-privileged instruction such as DCI. The typical instruction time is based on a 2 cycle instruction although many F8 instructions are one byte/one cycle instructions. The 6.0 μs max number represents the propagation delay through the peripheral device from $\overline{EXT\ INT}$ to $\overline{INT\ REQ}$ (interrupts from the timer do not incur this delay). Once the $\overline{INT\ REQ}$ is recognized, 22 cycles are required to stack the program counter and fetch the interrupt vector. One technique that can be used to minimize the maximum delay that would be incurred upon an interrupt is to constrain the instructions that are executed when the interrupt is expected. A method that would reduce the maximum delay from the interrupt striking to the execution of the first instruction of the service routine would be to put the processor in a branch on self loop (BR*). This essentially provides a wait for interrupt situation with the CPU running in a loop waiting for the interrupt.

INTERRUPT VECTOR FETCH

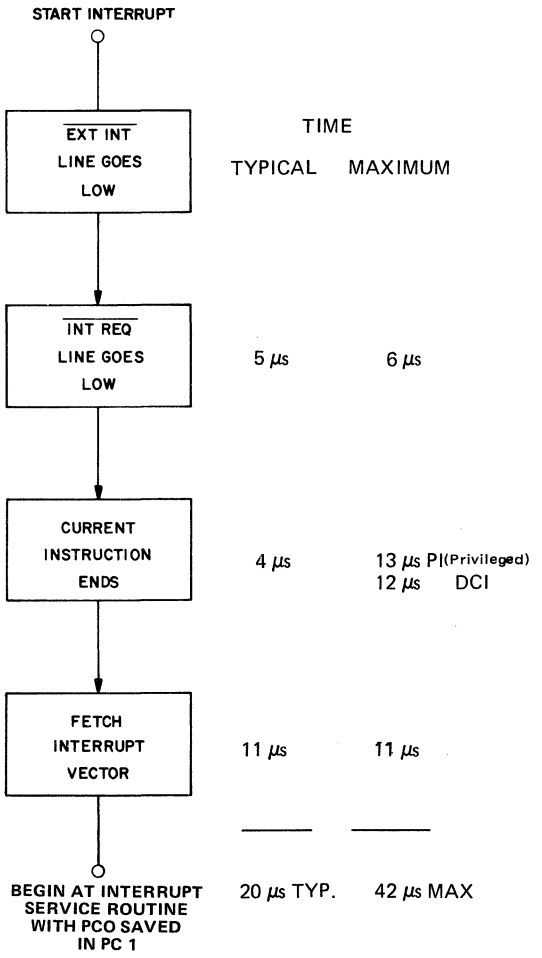


Figure 2

EXPANDING INTERRUPT INPUTS IN A MINIMUM SYSTEM

In a two-chip F8 microcomputer system (MK 3850 CPU and MK 3851 PSU) the system can be interrupted by either the timer in the PSU or the $\overline{EXT\ INT}$ line of the PSU. Thirty-two lines of bidirectional I/O are available and it may be desirable to have more than one input capable of interrupting

F8 Family

F8 SYSTEM INTERRUPT CONNECTION

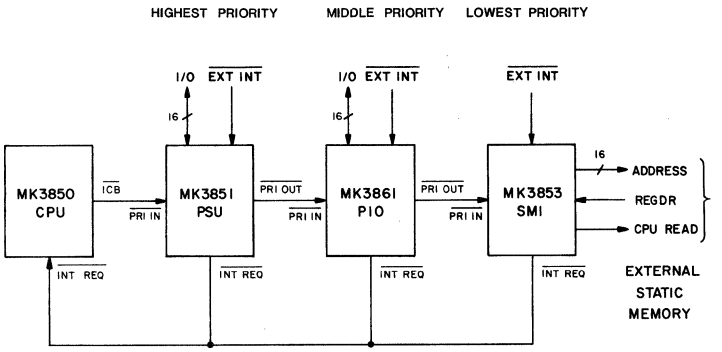


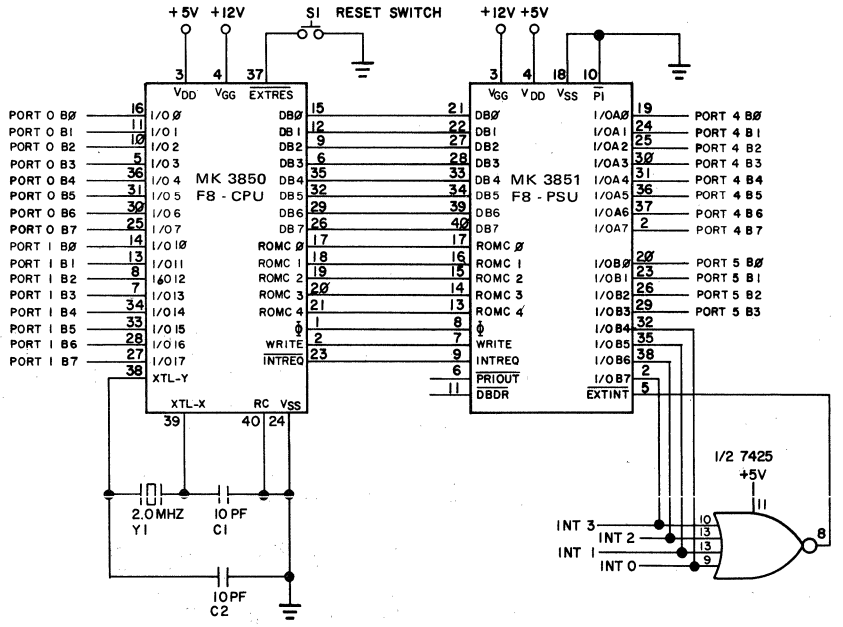
Figure 3

rupting the system. Figure 4 depicts this minimum F8 system, with four signals (INTO-INT3) capable of interrupting the system. The four external interrupting signals are defined active high and the presence of any one in the high state causes the output of the NOR gate to go low causing the interrupt. The interrupt service routine flowchart to locate the interrupting input is shown in Figure 5, with the actual program in Figure 6.

This service routine is entered with the interrupts automatically disabled at the CPU so that no further interrupts can occur until the interrupt is cleared by its service routine. The port containing the INT0-INT3 signals is loaded into the accumulator and tested to determine if bit 7 is low (a positive number). If bit 7 is low INT3 is active and the branch is taken to the service routine for INT3 (SERV3) (there is an inversion from the Port to the accumulator). If bit 7 is high a shift left one instruction is performed on the accumulator and it is again tested for bit 7 = 0 (bit 6 shifted). This process continues until all four of the interrupt lines have been tested. If an active interrupt bit has been found the proper service routine is branched to in order to service the active device and

F8 Family

EXPANSION OF INTERRUPT INPUTS IN A MINIMUM F8 SYSTEM



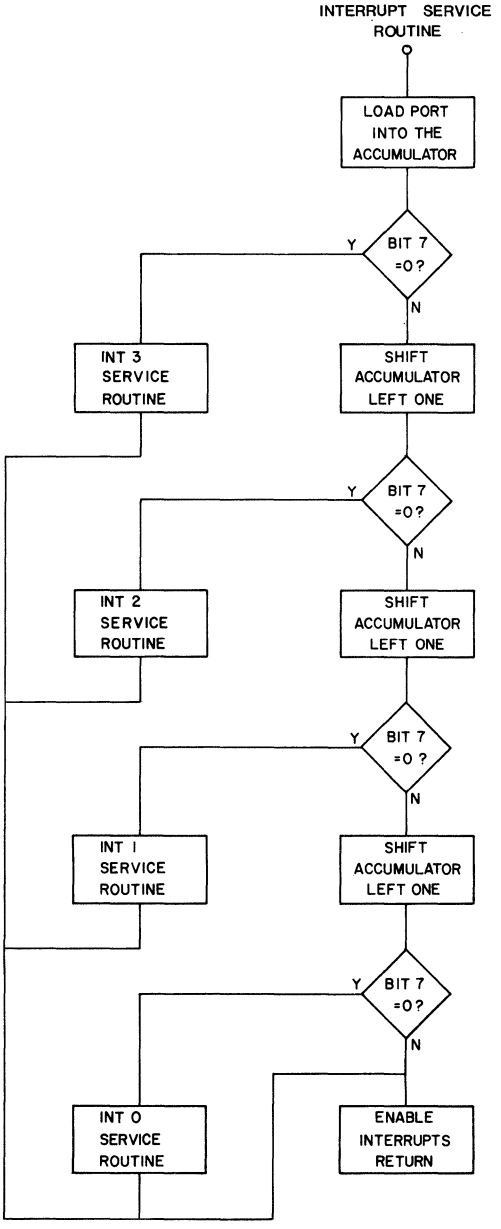
NOTE: MK 3870 Single Chip F8 will replace this two chip minimum system.

Figure 4

clear the interrupt. The routine ends by enabling the interrupts at the CPU and returning to the main program flow should no interrupt be found.

The additional time required to locate the active interrupt is a function of which interrupt is active due to the polling used. As shown in Figure 6, the additional delay to service interupts produced by polling varies from 15 μs for the highest priority device to 42 μs for the lowest priority device. To these times must be added the delays calculated earlier of 20 μs typical and 42 μs maximum which is required to get to the polling routine.

FLOWCHART OF INTERRUPT SERVICE ROUTINE



INTERRUPT SERVICE ROUTINE TO LOCATE INTERRUPTING DEVICE

CUMM μS	μS			
	8 INTSVC	INS	PORT	GET SIGNALS
INT 3 15	7	BP	SERV 3	INT 3 ACTIVE 7
	2	SL	1	NO, SHIFT LEFT
INT 2 24	7	BP	SERV 2	INT 2 ACTIVE 7
	2	SL	1	NO, SHIFT LEFT
INT 1 33	7	BP	SERV 1	INT 1 ACTIVE 7
	2	SL	1	NO, SHIFT LEFT
INT 0 42	7	BP	SERV 0	
	4	EI		ENABLE INTERRUPTS
	4	POP		RETURN

Figure 6

SINGLE CHIP MICROCOMPUTER

The MK 3870 Single Chip Microcomputer is the natural evolution of the F8 chip set. It will combine the functions of the 3850/3851 onto a single chip with the additions of another 1K bytes of ROM storage and an improved timer/interrupt structure. The techniques discussed in this application note apply also to the single chip F8 as it is software and hardware compatible with the multiple chip F8 family.